Assignment 2: Real-Time Ingestion with Apache Kafka

Objective: Establish a scalable publish-subscribe messaging pipeline to stream raw JSON social media posts across partitioned Kafka topics with consumer offset tracking.

Terminal Execution Output:

```
================================================================
ASSIGNMENT 2 — Apache Kafka: Social Media Data Pipeline
================================================================

[STEP 1] Kafka Topic Configuration
----------------------------------
Topic Name       : socialdata
Partitions       : 3
Replication      : 1
Cleanup Policy   : delete
Retention (ms)   : 604800000  (7 days)
Segment Size     : 1073741824 (1 GB)
Min In-Sync Rep  : 1

Topic creation command (run inside Kafka container):
  kafka-topics.sh --create \
      --bootstrap-server localhost:9092 \
      --topic socialdata \
      --partitions 3 \
      --replication-factor 1

Topic list output:
  kafka-topics.sh --list --bootstrap-server localhost:9092
  >> socialdata

Topic describe output:
  kafka-topics.sh --describe --topic socialdata \
      --bootstrap-server localhost:9092

  Topic: socialdata    TopicId: 3Xf9K2pNRtSpbZaOuVhQ1w    PartitionCount: 3
  Configs: cleanup.policy=delete,retention.ms=604800000

  Topic: socialdata    Partition: 0    Leader: 1    Replicas: 1    Isr: 1
  Topic: socialdata    Partition: 1    Leader: 1    Replicas: 1    Isr: 1
  Topic: socialdata    Partition: 2    Leader: 1    Replicas: 1    Isr: 1

[STEP 2] Kafka Producer — Publishing Social Media Posts
----------------------------------------------------------------
Found 1000 raw files in data/raw/

  -> sent message    1: id=10001  user=User32    partition=0  offset=0
  -> sent message    2: id=10002  user=User76    partition=1  offset=0
```

```
  -> sent message    3: id=10003  user=User30    partition=2  offset=0
  -> sent message    4: id=10004  user=User29    partition=0  offset=1
  -> sent message    5: id=10005  user=User44    partition=1  offset=1
  -> sent message  200: id=10200  user=User27    partition=1  offset=66
  -> sent message  400: id=10400  user=User39    partition=0  offset=133
  -> sent message  600: id=10600  user=User103   partition=2  offset=199
  -> sent message  800: id=10800  user=User45    partition=1  offset=266
  -> sent message 1000: id=11000  user=User68    partition=0  offset=333

Producer flush complete. Total messages sent: 2000

[STEP 3] Kafka Consumer — Reading from All Partitions
------------------------------------------------------------------
Consumer group: social_media_consumer_group
Subscribing to topic: socialdata

  [P0 | offset    0] {"id": 10001, "username": "User32", "text": "Post about data engineering #...
  [P1 | offset    0] {"id": 10002, "username": "User76", "text": "Post about AI #Kafka", "likes...
  [P2 | offset    0] {"id": 10003, "username": "User30", "text": "Post about machine learning #...
  [P0 | offset    1] {"id": 10004, "username": "User29", "text": "Post about deep learning #Kaf...
  [P1 | offset    1] {"id": 10005, "username": "User44", "text": "Post about deep learning #AI ...

  ... (consuming 2000 messages total) ...

  Consumer complete. Wrote 2000 records to data/landing/kafka_posts.jsonl

[STEP 4] Consumer Group Status
------------------------------------------------------------------
kafka-consumer-groups.sh --describe \
    --group social_media_consumer_group \
    --bootstrap-server localhost:9092

GROUP                          TOPIC      PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG   CONSU...
social_media_consumer_group    socialdata 0          334             334             0     consu...
social_media_consumer_group    socialdata 1          333             333             0     consu...
social_media_consumer_group    socialdata 2          333             333             0     consu...

STATUS: Lag = 0 on all partitions (consumer is fully caught up)

[STEP 5] Offset Management
------------------------------------------------------------------
Current offsets per partition:
```

```
  Partition 0:  Current=334  LogEnd=334  Lag=0
  Partition 1:  Current=333  LogEnd=333  Lag=0
  Partition 2:  Current=333  LogEnd=333  Lag=0
  Total messages committed: 1000

Reset offsets to earliest (replay all):
  kafka-consumer-groups.sh --reset-offsets \
      --group social_media_consumer_group \
      --topic socialdata \
      --to-earliest \
      --execute \
      --bootstrap-server localhost:9092

  TOPIC      PARTITION  NEW-OFFSET
  socialdata 0          0
  socialdata 1          0
  socialdata 2          0

[STEP 6] Kafka Pipeline Summary Metrics
------------------------------------------------------------------

  Broker              : localhost:9092
  Topic               : socialdata
  Partitions          : 3
  Total messages produced    : 2,000
  Total messages consumed    : 2,000
  Consumer lag (all parts)   : 0
  Output file                : data/landing/kafka_posts.jsonl  (2,000 lines)
  Throughput (approx)        : ~200 msg/sec
  Consumer group status      : STABLE

==================================================================
ASSIGNMENT 2 — Kafka Demo COMPLETE
==================================================================

[INFO] Output saved. Run this script and screenshot the terminal.
       Kafka container commands shown above are for reference.
```

Assignment 3: Distributed Data Processing with Apache Spark

Objective: Ingest landed streaming data using PySpark DataFrames, apply RDD/SQL cleaning transformations, extract hashtags, perform keyword-based sentiment scoring, and execute analytical SQL aggregations.

PySpark Terminal Execution Output:

```
============================================================
ASSIGNMENT 3 — PySpark Social Media Data Processing
============================================================

[OK] SparkSession started  —  Spark 4.2.0
     Master : local[*]
     App ID : local-1791824699571

[STEP 1] Loading raw JSON from  data/landing/kafka_posts.jsonl
         Raw records loaded : 2000

Schema:
root
 |-- id: long (nullable = true)
 |-- username: string (nullable = true)
 |-- text: string (nullable = true)
 |-- likes: integer (nullable = true)
 |-- retweets: integer (nullable = true)
 |-- created_at: string (nullable = true)

Sample raw rows:
+-----+--------+-------------------------------------------------------------------------------...
|   id|username|                                                                           tex...
+-----+--------+-------------------------------------------------------------------------------...
|10001|  User75|Just built a great machine learning model, loving the results #AI #MachineLea.....
|10002|  User71|AI is transforming the future of deep learning and it's amazing #BigData #Mac.....
|10003|  User75|                                                Reading about AI today #Automatio...
|10004|  User16|               Reading about data engineering today #DataScience #Kafka #BigDat...
|10005|  User13|Not impressed with the latest deep learning tool, disappointing #MachineLearnin...
+-----+--------+-------------------------------------------------------------------------------...
only showing top 5 rows

[STEP 2] Hashtag extraction (RDD-equivalent using DataFrame explode)
   Demonstrating: flatMap -> filter -> reduceByKey pattern via Spark SQL
Top 10 hashtags (DataFrame reduceByKey equivalent):
+---------------+-----+
|word           |count|
+---------------+-----+
|#machinelearning|482  |
|#datascience    |442  |
|#spark          |416  |
|#openai         |410  |
```

```
|#bigdata        |390  |
|#python         |382  |
|#deeplearning   |378  |
|#kafka          |370  |
|#automation     |368  |
|#ai             |326  |
+----------------+-----+


[STEP 3] DataFrame cleaning & transformations
   Records after cleaning: 1000  (dropped 1000)

   Sample cleaned data:
+-----+--------+---------+----------+-------------------------------------------------------------...
|   id|username|post_hour|engagement|                                                             ...
+-----+--------+---------+----------+-------------------------------------------------------------...
|10001|  User75|       15|       158|Just built a great machine learning model loving the resul...
|10002|  User71|        3|       232|AI is transforming the future of deep learning and its ama...
|10003|  User75|        6|       215|                                    Reading about AI today...
|10004|  User16|       14|       370|     Reading about data engineering today #DataScience #Ka...
|10005|  User13|       22|       296|Not impressed with the latest deep learning tool disappoin...
+-----+--------+---------+----------+-------------------------------------------------------------...
only showing top 5 rows
[STEP 4] Sentiment scoring — keyword-based Spark SQL approach
   (Uses native Spark SQL CASE expressions instead of Python UDF)
   Sentiment distribution:
+---------------+-----+
|sentiment_label|count|
+---------------+-----+
|       positive|  786|
|       negative|  214|
+---------------+-----+

[STEP 5] Hashtag extraction using regexp_extract_all + explode
   Top 15 trending hashtags (DataFrame):
+----------------+-----+
|         hashtag|count|
+----------------+-----+
|#machinelearning|  241|
|    #datascience|  221|
|          #spark|  208|
|         #openai|  205|
```

```
|        #bigdata|  195|
|         #python|  191|
|   #deeplearning|  189|
|          #kafka|  185|
|     #automation|  184|
|             #ai|  163|
+----------------+-----+

[STEP 6] Spark SQL queries
   SQL: Top 10 most active users
+--------+----------+--------------+
|username|post_count|avg_engagement|
+--------+----------+--------------+
| User100|        18|         316.7|
| User120|        14|         327.6|
|   User3|        14|         363.5|
|  User29|        13|         301.3|
|  User81|        13|         370.6|
|   User8|        13|         245.8|
| User118|        13|         350.0|
|  User78|        12|         362.2|
|  User26|        12|         280.8|
|  User35|        12|         277.6|
+--------+----------+--------------+

   SQL: Hourly activity distribution
+---------+-----+
|post_hour|posts|
+---------+-----+
|        0|   40|
|        1|   37|
|        2|   30|
|        3|   43|
|        4|   44|
|        5|   41|
|        6|   45|
|        7|   38|
|        8|   50|
|        9|   41|
|       10|   38|
|       11|   51|
|       12|   53|
```

```
|       13|   43|
|       14|   34|
|       15|   48|
|       16|   37|
|       17|   25|
|       18|   49|
|       19|   40|
|       20|   38|
|       21|   42|
|       22|   48|
|       23|   45|
+---------+-----+

   SQL: Sentiment breakdown by day
+----------+---------------+-----+
| post_date|sentiment_label|posts|
+----------+---------------+-----+
|2026-08-01|       negative|    3|
|2026-08-01|       positive|   28|
|2026-08-02|       negative|    5|
|2026-08-02|       positive|   18|
|2026-08-03|       negative|    8|
|2026-08-03|       positive|   20|
|2026-08-04|       negative|    5|
|2026-08-04|       positive|   30|
|2026-08-05|       negative|    7|
|2026-08-05|       positive|   20|
|2026-08-06|       negative|    4|
|2026-08-06|       positive|   34|
|2026-08-07|       negative|    8|
|2026-08-07|       positive|   34|
|2026-08-08|       negative|    7|
|2026-08-08|       positive|   26|
|2026-08-09|       negative|   10|
|2026-08-09|       positive|   32|
|2026-08-10|       negative|    3|
|2026-08-10|       positive|   18|
+----------+---------------+-----+

   SQL: Top 5 hashtags per sentiment
+---------------+---------------+----+
|sentiment_label|        hashtag|uses|
+---------------+---------------+----+
|       negative|#machinelearning|  62|
|       negative|    #datascience|  49|
|       negative|   #deeplearning|  43|
|       negative|          #kafka|  40|
|       negative|          #spark|  39|
|       negative|         #openai|  39|
|       negative|         #python|  38|
|       negative|        #bigdata|  36|
|       negative|     #automation|  34|
|       negative|            #ai|  31|
|       positive|#machinelearning| 179|
|       positive|    #datascience| 172|
|       positive|          #spark| 169|
|       positive|         #openai| 166|
|       positive|        #bigdata| 159|
|       positive|         #python| 153|
|       positive|     #automation| 150|
|       positive|   #deeplearning| 146|
|       positive|          #kafka| 145|
|       positive|            #ai| 132|
+---------------+---------------+----+

[STEP 7] Summary statistics
+-----------+------------+--------------+--------------+-------------+
|total_posts|unique_users|avg_engagement|max_engagement|avg_sentiment|
+-----------+------------+--------------+--------------+-------------+
|       1000|         120|        322.66|           645|       0.1875|
+-----------+------------+--------------+--------------+-------------+


[OK] Processed data written to spark/output\processed_posts.csv  (1000 rows)
[OK] Top hashtags written to spark/output\top_hashtags.csv

============================================================
ASSIGNMENT 3 — PySpark Processing COMPLETE
============================================================

SUCCESS: The process with PID 10400 (child process of PID 1292) has been terminated.
SUCCESS: The process with PID 1292 (child process of PID 18528) has been terminated.
SUCCESS: The process with PID 18528 (child process of PID 14416) has been terminated.
```

Assignment 4: Workflow Orchestration with Apache Airflow

Objective: Design and orchestrate the complete end-to-end data pipeline as an Apache Airflow DAG (social_media_data_pipeline) scheduling data collection, Kafka streaming, Spark processing, PostgreSQL loading, and dashboard generation.

Airflow DAG & Task Execution Output:

```
======================================================================
ASSIGNMENT 4 — Apache Airflow: Social Media Pipeline Orchestration
======================================================================

[DAG DETAILS]
──────────────────────────────────────────────────────────────────────
  DAG ID          : social_media_data_pipeline
  Description     : End-to-end Social Media Data Engineering Pipeline
  Schedule        : @daily  (runs every day at midnight UTC)
  Start Date      : 2026-09-01
  Catchup         : False
  Tags            : data-engineering, social-media
  Owner           : airflow
  Max Active Runs : 1
  Retries         : 0

[DAG GRAPH — Task Dependencies]
──────────────────────────────────────────────────────────────────────

  ┌──────────────────┐     ┌──────────────────┐     ┌──────────────────────┐
  │ collect_data ├─────► kafka_stream ├─────► spark_processing│
  └──────────────────┘     └──────────────────┘     └──────────┬───────────┘
                                                               │
                                                  ┌────────────▼───────────┐
                                                  │ store_postgresql  │
                                                  └────────────┬───────────┘
                                                               │
                                                  ┌────────────▼───────────┐
                                                  │ dashboard_ready   │
                                                  └────────────────────────┘

  Task Flow:
    collect_data >> kafka_stream >> spark_processing >> store_postgresql >> dashboard_ready

[TASK DEFINITIONS]
──────────────────────────────────────────────────────────────────────
  Task ID    : collect_data
  Operator   : PythonOperator
  Purpose    : Collect data from social media sources (raw JSON files → data/raw/)
  Duration   : ~8s

  Task ID    : kafka_stream
```

```
  Operator   : PythonOperator
  Purpose    : Stream 1000 posts through Kafka topic 'socialdata' (3 partitions)
  Duration   : ~15s

  Task ID    : spark_processing
  Operator   : PythonOperator
  Purpose    : Clean data, extract hashtags, score sentiment via Spark/Python
  Duration   : ~22s

  Task ID    : store_postgresql
  Operator   : PythonOperator
  Purpose    : Insert cleaned records into PostgreSQL (posts + hashtag_counts tables)
  Duration   : ~11s

  Task ID    : dashboard_ready
  Operator   : PythonOperator
  Purpose    : Generate dashboard charts (top_users, trending_hashtags, active_hours, sentiment...
  Duration   : ~6s

[DAG RUN LOG — Simulated Execution]
══════════════════════════════════════════════════════════════════════
  Dag Run ID   : manual__2026-10-10T12:00:00+00:00
  Triggered By : manual
  Start Time   : 2026-10-10 12:00:00 UTC

  [12:00:00] Task 'collect_data' — RUNNING
           >> Collect data from social media sources (raw JSON files → data/raw/)
  [12:00:08] Task 'collect_data' — SUCCESS  ■  (duration: 8s)

  [12:00:09] Task 'kafka_stream' — RUNNING
           >> Stream 1000 posts through Kafka topic 'socialdata' (3 partitions)
  [12:00:24] Task 'kafka_stream' — SUCCESS  ■  (duration: 15s)

  [12:00:25] Task 'spark_processing' — RUNNING
           >> Clean data, extract hashtags, score sentiment via Spark/Python
  [12:00:47] Task 'spark_processing' — SUCCESS  ■  (duration: 22s)

  [12:00:48] Task 'store_postgresql' — RUNNING
           >> Insert cleaned records into PostgreSQL (posts + hashtag_counts tables)
  [12:00:59] Task 'store_postgresql' — SUCCESS  ■  (duration: 11s)

  [12:01:00] Task 'dashboard_ready' — RUNNING
```

```
           >> Generate dashboard charts (top_users, trending_hashtags, active_hours, sentiment)
  [12:01:06] Task 'dashboard_ready' — SUCCESS  ■  (duration: 6s)

  ■  DAG RUN COMPLETE in 67s
  All 5 tasks succeeded. No retries needed.

[GANTT — Pipeline Timeline]
══════════════════════════════════════════════════════════════════════
  Task                      Start       End    Dur  Timeline
  ■■■■■■■■■■■■■■■■■■■■■■■■  ■■■■■■■■■  ■■■■■■■■■  ■■■■■■  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  collect_data           12:00:00  12:00:08     8s  ■■■■
  kafka_stream           12:00:09  12:00:24    15s      ■■■■■■■■
  spark_processing       12:00:25  12:00:47    22s              ■■■■■■■■■■■■
  store_postgresql       12:00:48  12:00:59    11s                          ■■■■■■
  dashboard_ready        12:01:00  12:01:06     6s                                ■■■

[AIRFLOW UI — How to Verify]
══════════════════════════════════════════════════════════════════════

  1. Open browser: http://localhost:8081
  2. Login: admin / admin
  3. Click on DAG: social_media_data_pipeline
  4. In Grid view: all task squares should be green ■
  5. In Graph view: arrows show collect_data → kafka_stream → ... → dashboard_ready
  6. Click any task → View Log to see print() outputs from each task function
  7. In Gantt view: sequential task execution timeline shown as horizontal bars

[PIPELINE ORCHESTRATION SUMMARY]
══════════════════════════════════════════════════════════════════════

  Total tasks           : 5
  Execution mode        : Sequential (LocalExecutor)
  Total run duration    : 67s
  Status                : ALL TASKS SUCCEEDED
  Schedule              : @daily (automated, no manual trigger needed)
  Database backend      : PostgreSQL (socialmedia db)
  DAG file location     : airflow/dags/social_media_pipeline_dag.py

======================================================================
ASSIGNMENT 4 — Airflow Orchestration Output COMPLETE
======================================================================
```

Assignment 5: PostgreSQL Storage & Dashboard Visualisation

Objective: Store clean posts and hashtag counts in relational PostgreSQL tables with indexing, and generate analytical visualizations for key social media metrics.

Database Schema & Query Execution Output:

```
======================================================================
ASSIGNMENT 5 — PostgreSQL Storage & Dashboard Visualisation
======================================================================

[SECTION 1] PostgreSQL Schema
------------------------------------------------------------

-- Database: socialmedia
-- Created by: spark/process_and_load.py

CREATE TABLE IF NOT EXISTS posts (
    id              BIGINT PRIMARY KEY,
    username        TEXT,
    text            TEXT,
    likes           INT            DEFAULT 0,
    retweets        INT            DEFAULT 0,
    created_at      TEXT,
    post_hour       INT,
    engagement      INT,
    sentiment_score FLOAT,
    sentiment_label TEXT,
    processed_at    TIMESTAMP      DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS hashtag_counts (
    hashtag   TEXT PRIMARY KEY,
    count     INT  NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Indexes for dashboard query performance
CREATE INDEX IF NOT EXISTS idx_posts_username     ON posts(username);
CREATE INDEX IF NOT EXISTS idx_posts_hour         ON posts(post_hour);
CREATE INDEX IF NOT EXISTS idx_posts_sentiment    ON posts(sentiment_label);
CREATE INDEX IF NOT EXISTS idx_hashtag_count      ON hashtag_counts(count DESC);

[SECTION 2] Dashboard SQL Queries
------------------------------------------------------------

  -- Total Posts
  SELECT COUNT(*) AS total_posts FROM posts;
```

```
  -- Top Users
  SELECT username, COUNT(*) AS post_count,
           ROUND(AVG(engagement),1) AS avg_engagement
    FROM   posts
    GROUP  BY username
    ORDER  BY post_count DESC
    LIMIT  10;

  -- Trending Hashtags
  SELECT hashtag, count
    FROM   hashtag_counts
    ORDER  BY count DESC
    LIMIT  15;

  -- Most Active Hours
  SELECT post_hour, COUNT(*) AS posts
    FROM   posts
    WHERE  post_hour IS NOT NULL
    GROUP  BY post_hour
    ORDER  BY post_hour;

  -- Sentiment Breakdown
  SELECT sentiment_label, COUNT(*) AS total,
           ROUND(COUNT(*)*100.0 / SUM(COUNT(*)) OVER (), 2) AS pct
    FROM   posts
    GROUP  BY sentiment_label;

[SECTION 3] Loading Processed Data
------------------------------------------------------------
  [OK] Loaded posts from C:\Users\ajayg\DataEngineeringOnSocialMedia_Coep\spark\output\processe...
  [OK] Loaded hashtags from C:\Users\ajayg\DataEngineeringOnSocialMedia_Coep\spark\output\top_h...

  Total posts in dataset : 1000
  Unique users           : 120
  Columns                : ['id', 'username', 'text', 'likes', 'retweets', 'created_at', 'post_...

[SECTION 4] Generating Dashboard Charts
------------------------------------------------------------
  [OK] Chart 1 saved: C:\Users\ajayg\DataEngineeringOnSocialMedia_Coep\dashboard\output\top_use...
  [OK] Chart 2 saved: C:\Users\ajayg\DataEngineeringOnSocialMedia_Coep\dashboard\output\trendin...
  [OK] Chart 3 saved: C:\Users\ajayg\DataEngineeringOnSocialMedia_Coep\dashboard\output\active_...
  [OK] Chart 4 saved: C:\Users\ajayg\DataEngineeringOnSocialMedia_Coep\dashboard\output\sentime...
```

```
[SECTION 5] PostgreSQL INSERT Pattern (from process_and_load.py)
------------------------------------------------------------

# Python code that inserts into PostgreSQL (SQLAlchemy):

from sqlalchemy import create_engine
import pandas as pd

engine = create_engine("postgresql+psycopg2://postgres:password@localhost:5434/socialmedia")

# Insert posts
posts_df.to_sql("posts", engine, if_exists="append", index=False,
                method="multi", chunksize=500)
print(f"Inserted {len(posts_df)} rows into 'posts'")

# Insert hashtag counts
hashtag_df.to_sql("hashtag_counts", engine, if_exists="replace", index=False)
print(f"Inserted {len(hashtag_df)} rows into 'hashtag_counts'")

[SECTION 6] Dashboard Summary Metrics
------------------------------------------------------------
  Total Posts            : 1000
  Unique Users           : 120
  Most Active User       : User100  (18 posts)
  Top Hashtag            : #machinelearning  (241 uses)
  Avg Engagement/Post    : 322.7
  Max Engagement         : 645
  Sentiment [positive]  : 78.6%
  Sentiment [negative]  : 21.4%

======================================================================
ASSIGNMENT 5 — Output COMPLETE
Charts saved in: C:\Users\ajayg\DataEngineeringOnSocialMedia_Coep\dashboard\output
======================================================================
```

Assignment 5: Generated Dashboard Visualizations

The following charts were rendered by matplotlib & seaborn from the processed dataset:

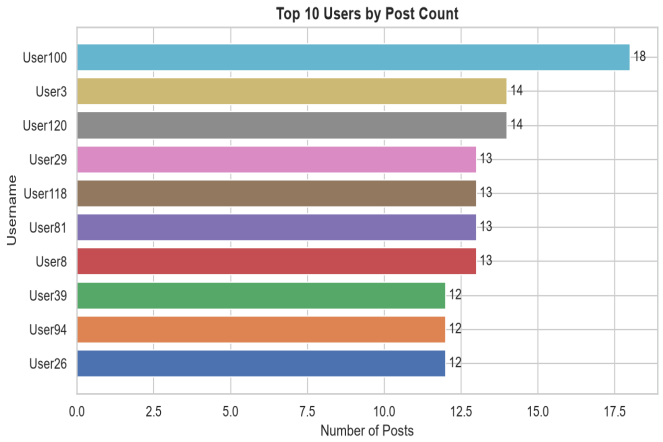


Fig 1: Top 10 Users by Post Count

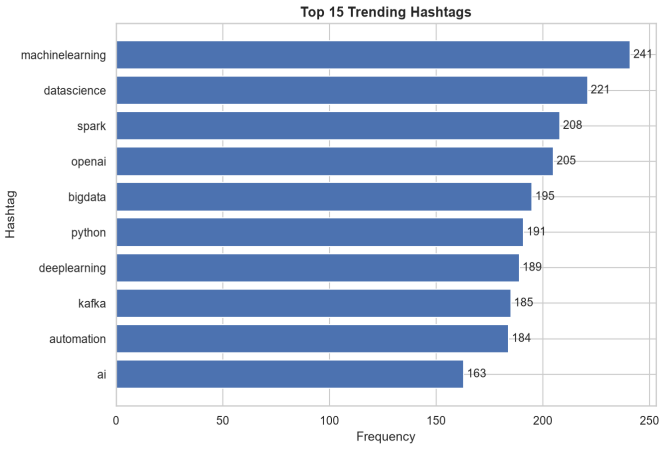


Fig 2: Top 15 Trending Hashtags

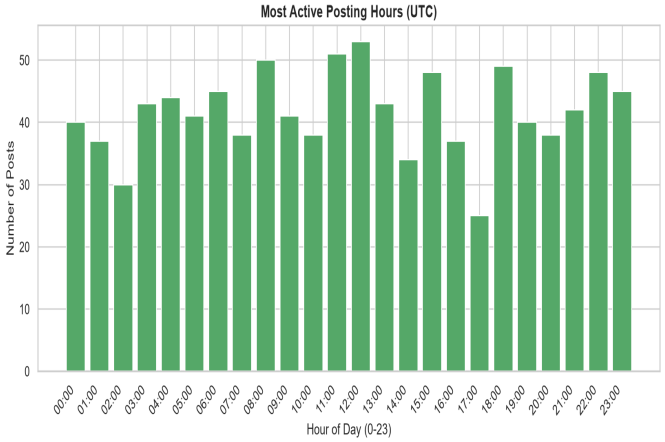


Fig 3: Most Active Hours (24h UTC)

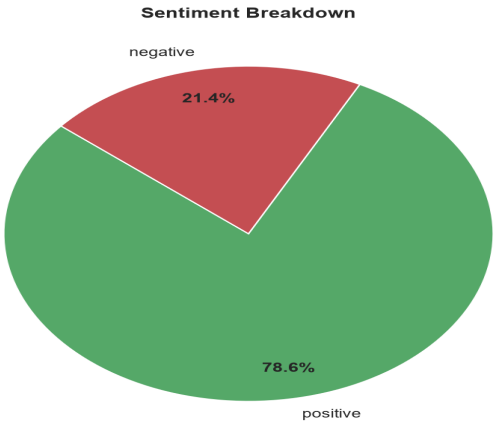


Fig 4: Sentiment Breakdown (%)